Introduction

The QDET web application has been built using the framework Django. This framework is based on the Python programming language. Is follows the typical Model-View-Controller (MVC) architecture, although the naming of the files is different. To create the tables of the database, we use python classes. These are saved in the Models directory, and correspond to the Model in the MVC architecture. The Controllers of the MVC are, in Django, named Views, and the Views of the MVC are named Templates.

Django is a framework that is structured by “Apps”. Each app is dedicated to a specific feature, or set of features. It allows to create different independent “blocs”, and to put them together to build up a web application with different features and facets.

The Project’s Structure:

We have built two separate Django Apps: “directory” and “login”. Both follow the same structure:

- The models directory: contains all the classes building the database. To have more information about the database structure, please refer to the provided UML diagram.
- The views directory: contains all the python functions which will handle the user’s requests.
- The templates directory: contains all the HTML files that the end user will receive.
- The migrations directory: contains all the automatically generated migrations of the database.

Some other files are present in each app :

- urls.py: contains all the existing URL that any user can access, each URL points to a view function.
- forms.py: file use to specify which model and which of its attributes can be inputted by a user inside a form in the front-end.
- admin.py: file used to register a database table in the Administration Panel.

Our project contains other folders that are not apps:

- media: folder used to save all the content uploaded by users that are not saved in the database. For our project, we can expect it to contain pickle files, as well as images for the histograms.
- static: folder use to save static files used for the front-end. This directory contains the CSS, JavaScript, libraries, text-font, and others.
- templates: This folder contains the base.html template. This HTML file is the base file upon which all other HTML file are built on-top of.
- modules: A set of custom built python functions that are needed for the backend. They are mostly related with the IA models: training, evaluating, saving data…

- Project: a directory automatically generated by Django. It contains files needed for the proper behavior of the framework. The most relevant file is the settings.py present in that directory.

Main functionalities:

Login app:

This app handles different important and required functionalities: login and registration.
It contains various views, allowing users to login, register, reset their password, activate their accounts, and others.
For development purposes, on the register.py file, we do not verify the email of newly registered users. This feature is meant for production purposes, although the code of this app is production ready. In the login/views/ directory, we do not make use of either the email_activation.py file (handles the email sending to verify the email), and the deactivate_account.py. This file would be used to allow users to "delete" (deactivate) their accounts. This feature has not been implemented (more about it in the Technical Debt section).

Directory app:

This is where the magic happens. This app is the main app implementing all the unique functionalities of this web application. Let's break it down in different sections:

directory/models:
The set of classes which together build the database of our project. The user-uploaded MCQs get saved in the Context, Questions, and Answers according to what they correspond to. Users can choose to train a model. Should they save it, its parameters are defined in the Trained_Model.py file. With a trained model, users can run predictions of their MCQs. The results will be saved thanks to the Prediction.py file. Finally, the Histogram and Statistic classes will be saving the results of the evaluation of the predictions.
Note that there is no User model anywhere in the project. This is because the users are handled by the Django framework. There is therefore no need to define that model explicitly.

directory/views:
These files are basically the backend of the web application. In the views, we obtain the request of a user. We can make queries to the database to obtain specific data, run calculations, or in reality many different things, before we finally load up an HTML file (templates), and send the response to the user.

The index.py file is the entry point for users. If the user is not logged in, they will be shown a different HTML file from if they were logged in. If the user is logged in, the index.py file serves as the backend for the dashboard.

Our webapp allows users to upload, visualise, edit, and delete MCQs. There are different views for each of these functionalities. There are two ways of uploading a new MCQ: bulk-import through the upload_csv.py file, and upload an individual question through the combination of new_context.py (users create or select a context), and then the new_question.py (where users then input both the question and its answers). Once uploaded, users can see all of their uploaded questions with the user_questions.py view, or choose to see all the details of one question with the single_question.py view.
They can chose to update an individual question, with the update_question.py file, or delete it with the delete_question.py file.

The second block of our webapp is in relation with the AI models. Each feature also has its dedicated view. First of, users can choose to tain and save an AI model, with the train_select.py file. This file does not do the training itself. Instead, it takes in the request of the user, and calls another file: modules/train_model.py. This second file is the one that plugs with the text2props library and actually does the training.
Once a model has been trained, users can evaluate or estimate the values of the different latent-traits of their own questions. To run an evaluation, the view directory/views/evaluate_select.py comes into play. Just like with the training, this view does not run the AI predictions itself. Instead, it takes the request from the user, loads the set of questions, and calls the modules/evaluate.py file which plugs into the text2props library and runs the evaluation.
Once the evaluation has ran, other files inside the modules directory are called: histogram_creation.py and statistical_metrics.py. These will generate the histogram and save it to the database, as well as compute the statistical data of the prediction, and also save them to the database.
To display the results of an evaluation, the view directory/views/evaluations.py is used.

Templates:
Whilst the views correspond to the backend and treat the request of the users, the Templates are the files that are actually returned to the user. They correspond to the Front-End of the webapp. Each view as a corresponding template, named in same way as the view.
The only exception is the index. There are two templates for this view. The first one: index.html is the one loading the dashboard. As the dashboard is proper to logged-in users, we defined another template: index_not_logged_in.html, which displays a page for non-logged-in users.

Technical Debt:

Some precise, great to have but not crucial features have been left out due to a lack of development time. For the time being, users cannot:
- Modify or delete any of their trained models
- Delete any predictions or histograms
- Delete a context; although by deleting all the question tied to a context, the context will become visually hidden

- Delete their accounts

Another important point to mention is that any modification in the data-set of a user (a question is modified / added / removed…) will not automatically update the already ran predictions (histograms and statistical data). Users will have to re-run evaluations to obtain the new, up to date information.